



PennyPal
Fresh All Along

Project Documentation

A cross-platform personal finance app for students

Item	Details
Project name	PennyPal
Theme	NextGen BudgetBee
Category	Multi-Platform App Computing
Platforms	Android phones, iPhone / iPad, tablets, web browsers
Technology	Flutter 3.27.4, Dart 3.6, SQLite, Firebase (optional), Google Gemini (optional)
Document version	1.0
Date	24 September 2026
Prepared by	_____

Table of Contents

1. Problem Definition.....	3
1.1 Background.....	3
1.2 Problem Statement.....	3
1.3 Objectives.....	3
1.4 Scope.....	3
1.5 User Roles and Permissions.....	4
1.6 Constraints.....	4
2. Design Specifications.....	6
2.1 Technology Stack.....	6
2.2 System Architecture.....	6
2.3 Module Design.....	8
2.4 Site Map.....	9
2.5 User Interface Design.....	9
2.6 Key Business Rules and Algorithms.....	9
2.7 Security and Privacy Design.....	10
2.8 Non-Functional Requirements.....	11
2.9 Requirements Traceability.....	11
3. Diagrams.....	13
3.1 Data Flow Diagram – Level 0 (Context).....	13
3.2 Data Flow Diagram – Level 1.....	13
3.3 Flowcharts.....	13
3.4 Entity Relationship Diagram.....	20
4. Database Design.....	21
4.1 Overview.....	21
4.2 Table Definitions.....	21
4.3 Relationships.....	25
4.4 Cloud Firestore Structure (optional).....	25
4.5 SQL Scripts Submitted.....	25
5. Test Data and Testing.....	27
5.1 Test Data.....	27
5.2 Test Cases.....	28
5.3 Automated Tests.....	29
6. Project Installation Instructions.....	31
6.1 Prerequisites.....	31
6.2 Install the APK on an Android phone.....	31
6.3 Run from the source code.....	31
6.4 Optional: enable the Gemini chatbot.....	31
6.5 Optional: enable Firebase cloud sync.....	31
6.6 Load the database scripts (optional).....	31
7. User Credentials.....	32
8. Assumptions.....	33
9. Acknowledgements.....	34
9.1 Third-party libraries and resources.....	34
9.2 AI tools used.....	34
Appendix A. Screenshots.....	35
A.1 Student app.....	35
A.2 Admin portal.....	38
A.3 Tablet / web layout.....	39

1. Problem Definition

1.1 Background

Many students start receiving money for the first time through allowances, scholarships, internships or part-time jobs, usually without any habit of recording what they spend. Small daily costs such as food, transport, subscriptions, entertainment and study materials add up quickly and leave little for necessities or savings. Notebooks and spreadsheets are slow to maintain on the go, so most students stop tracking after a few days and only notice a problem when the money has already run out.

1.2 Problem Statement

Students need a quick, simple and trustworthy way to:

- record income and expenses in a few taps, even without an internet connection;
- plan how much they can spend each month and in each category;
- be warned before, not after, they overspend;
- save steadily towards personal goals and see their progress;
- understand their spending habits and learn basic money-management skills.

Existing banking or investment apps are too complex for beginners, are often tied to a particular bank, and are not designed to teach. PennyPal addresses this gap with a focused, beginner-friendly tool that works the same on Android, iOS, tablets and the web.

1.3 Objectives

Table 1: Project objectives

#	Objective	How PennyPal meets it
O1	Make expense tracking fast	Add income/expense from any screen with a floating Add button; smart category suggestion fills the category automatically.
O2	Support budgeting	Monthly overall budget plus category limits with a configurable alert threshold (default 80%).
O3	Prevent overspending	Real-time alerts the moment a threshold or limit is crossed, shown on screen, stored in Notifications and sent as a device notification.
O4	Encourage saving	Savings goals with progress rings, milestones (25/50/75/100%), estimated completion date and a behind-schedule warning.
O5	Build financial literacy	Learning hub with short lessons and the "Ask Penny" chatbot for budgeting questions.
O6	Give insight	Dashboard summary, personalised tips, category charts, 6-month trend and exportable reports.
O7	Work anywhere	One Flutter codebase for Android, iOS, tablets and web; offline-first local database with cloud sync.
O8	Allow administration	Admin portal for students, learning content, support queries, feedback, settings and analytics.

1.4 Scope

In scope

- Student registration, login and logout; pre-configured administrator login.
- Income and expense management with categories, search and filters, receipt photos.
- Monthly and category budgets, real-time alerts and notifications.

- Savings goals with contributions, milestones and goal history.
- Dashboard, spending reports, charts and CSV export.
- Learning hub, About, Feedback, Contact Support and the AI chatbot.
- Optional AI features: automatic expense categorisation and the Gemini-powered chatbot.
- Offline data entry with automatic synchronisation to the cloud when available.
- Admin portal: manage students, monitor activity, analytics report, learning content, support and settings.

Out of scope (intentionally excluded)

PennyPal is a finance-learning and expense-management tool. As required by the specification it does not:

- connect to bank accounts, store card details or perform payments or transfers;
- act as a bank, digital wallet, payment gateway or investment platform;
- give investment, loan or professional financial advice. The chatbot declines such questions and every lesson and AI answer is labelled as educational guidance only.

1.5 User Roles and Permissions

Table 2: Role-based permissions

Permission	Student	Administrator
Register a new account	Yes	No (account is pre-configured)
Log in / log out	Yes	Yes
Manage own income, expenses, budgets, goals, categories	Yes	No
View own dashboard, reports, notifications	Yes	No
Use chatbot and learning hub	Yes	No
Submit feedback and support queries	Yes	No
View all students, activate / deactivate / delete accounts	No	Yes
View platform statistics and export analytics report	No	Yes
Create, edit, hide and delete learning content	No	Yes
Reply to support queries and read feedback	No	Yes
Change app settings (chatbot on/off, default alert threshold, support email)	No	Yes

1.6 Constraints

Table 3: Constraints from the specification

Constraint	How it is handled
Real-time processing	Budget usage is recalculated in memory each time an expense is saved; alerts appear immediately without a server round-trip.
Cross-platform compatibility	Single Flutter codebase; responsive layouts switch from a bottom navigation bar (phones) to a navigation rail (tablets and web, width 840 px or more).
Security and privacy	PBKDF2-hashed passwords, per-user data access, HTTPS for cloud and AI calls, Firestore security rules, no bank or card data collected.

Constraint	How it is handled
Offline support	All data is written to a local SQLite database first and queued for synchronisation when the device is back online.
AI response latency	The chatbot shows a "Penny is thinking..." indicator and gives up after 10 seconds, falling back to the built-in rule-based advisor.

2. Design Specifications

2.1 Technology Stack

Table 4: Technology stack

Layer	Technology	Purpose
Framework	Flutter 3.27.4, Dart 3.6	One codebase for Android, iOS, tablets and web
State management	provider	Session state and per-student finance state shared across screens
Local database	SQLite via sqflite (mobile) and sqflite_common_ffi_web (web)	Offline storage and sync queue
Cloud (optional)	Firebase Authentication, Cloud Firestore	Account sign-in and cloud backup of records
AI chatbot	Google Gemini API (key from Google AI Studio) over HTTPS	Budgeting answers; offline rule-based fallback
AI categorisation	string_similarity (Dice coefficient)	On-device category suggestion that tolerates typos
Charts	fl_chart	Donut and bar charts
Device features	image_picker, flutter_local_notifications, connectivity_plus, share_plus, path_provider	Receipt photos, notifications, connectivity detection, CSV export
Security	crypto (PBKDF2-HMAC-SHA256), uuid	Password hashing, collision-free record IDs
Formatting	intl	Currency and date formatting
Build / CI	GitHub Actions	Runs analysis and tests, builds APK and web app on each push

2.2 System Architecture

PennyPal follows a layered, offline-first architecture. The user interface never talks to the database directly: screens read and update state objects, state objects call services and repositories, and repositories are the only code that reads or writes SQLite. Every write also records an entry in a sync queue, which a background service pushes to Cloud Firestore when the student is signed in and online.

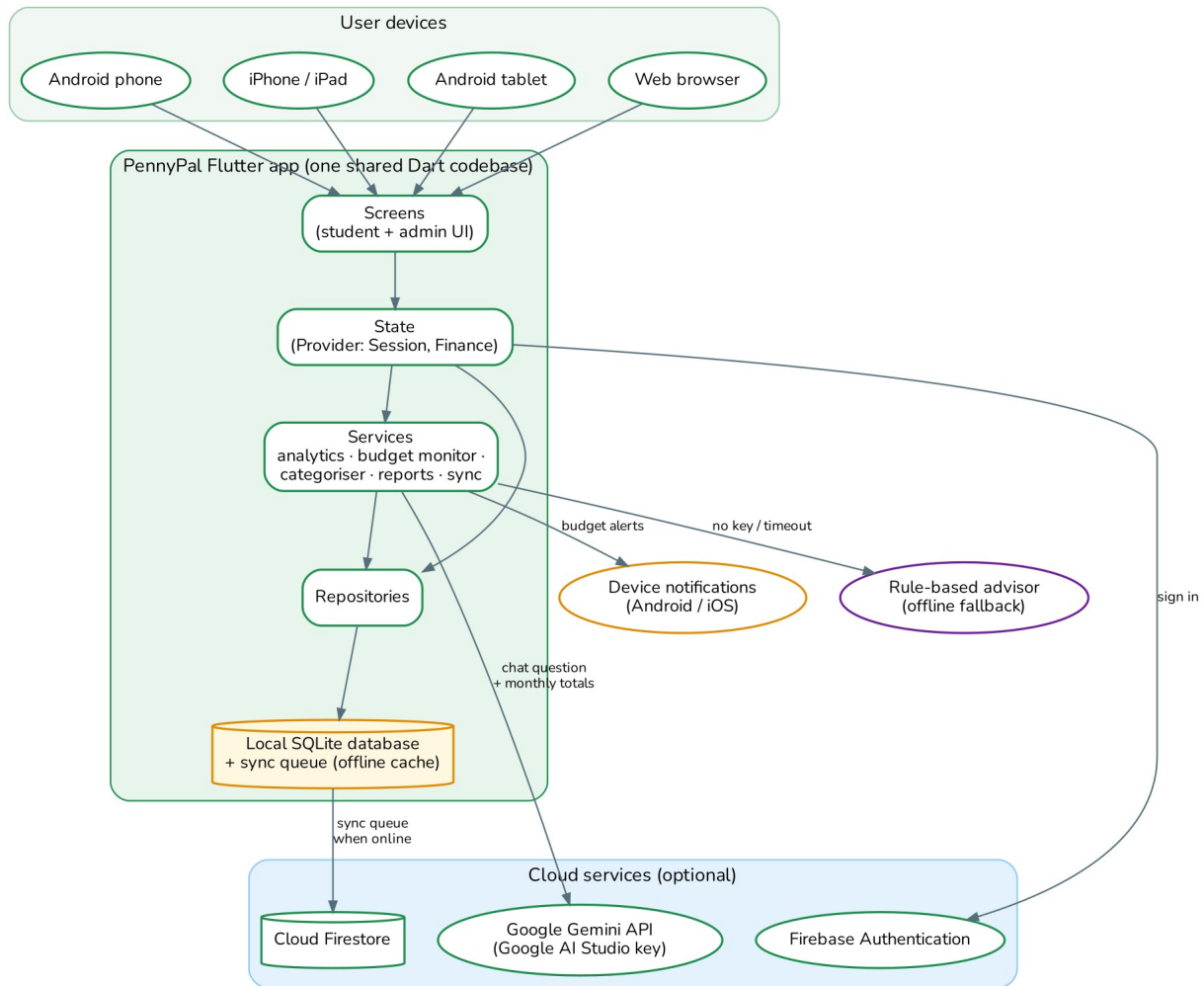


Figure 1: PennyPal system architecture

Table 5: Architecture layers

Layer	Responsibility
Screens	Student screens (dashboard, history, budget, goals, reports, chatbot, learning, feedback, support, profile) and admin screens (overview, students, content, support, settings).
State	SessionState keeps the logged-in user and profile; FinanceState keeps the student's transactions, budgets, goals and notifications in memory for fast screens.
Services	Analytics (totals, trends, insights), budget monitor (alerts), expense categoriser, chatbot, report builder, notification and sync services.
Repositories	Auth, finance, engagement (lessons, support, feedback, notifications, settings) and admin queries against SQLite.
Local database	SQLite schema created from the submitted SQL script; seeded with default categories, admin account, lessons and demo data.
Cloud (optional)	Firebase Auth for sign-in, Firestore for backup, Gemini for AI answers. The app remains fully usable without them.

2.3 Module Design

Table 6: Modules

Module	Main functions	Screens
Authentication	Register with validation and duplicate check, login, persistent session, logout, change password, account deactivation check	Login, Register, Profile
Dashboard	Available balance, total income/expenses, saved in goals, budget status, quick actions, budget progress, tips, spending chart, recent transactions, empty-state messages	Home
Income & expenses	Add, view, edit, delete; income source; expense category; payment mode; receipt photo; smart category suggestion	Transaction form, details sheet
Transaction history	Search text; filter by type, category and date range; daily grouping; swipe to delete	History
Categories	8 default categories plus custom categories with icons	Categories
Budget management	Monthly budget, category limits, alert threshold, remaining / overspent, copy last month, month navigation	Budget
Alerts & notifications	Threshold and limit alerts, goal milestones, support replies; in-app list and device notifications	Notifications
Savings goals	Create/edit/delete, contributions, progress %, remaining, estimated completion, milestones, archive to history	Goals, Goal form
Reports	Period presets and custom range, totals, daily average, category breakdown, 6-month trend, CSV export, report history	Spending reports
Learning hub	Topic filter, lesson cards, lesson detail with disclaimer	Learning hub, Lesson
Chatbot	Gemini answers with student context, 10 s timeout, rule-based fallback, suggestion chips	Ask Penny
Feedback & support	Feedback with rating; support query with status and admin reply	Feedback, Contact support
About & privacy	Purpose, disclaimer, privacy notice, acknowledgements	About & privacy
Admin portal	Statistics, student management, content management, support replies, feedback, settings, analytics export	Overview, Students, Content, Support, Settings
Sync	Offline queue, connectivity detection, retry, per-user scoping	Sync indicator, Profile

2.4 Site Map

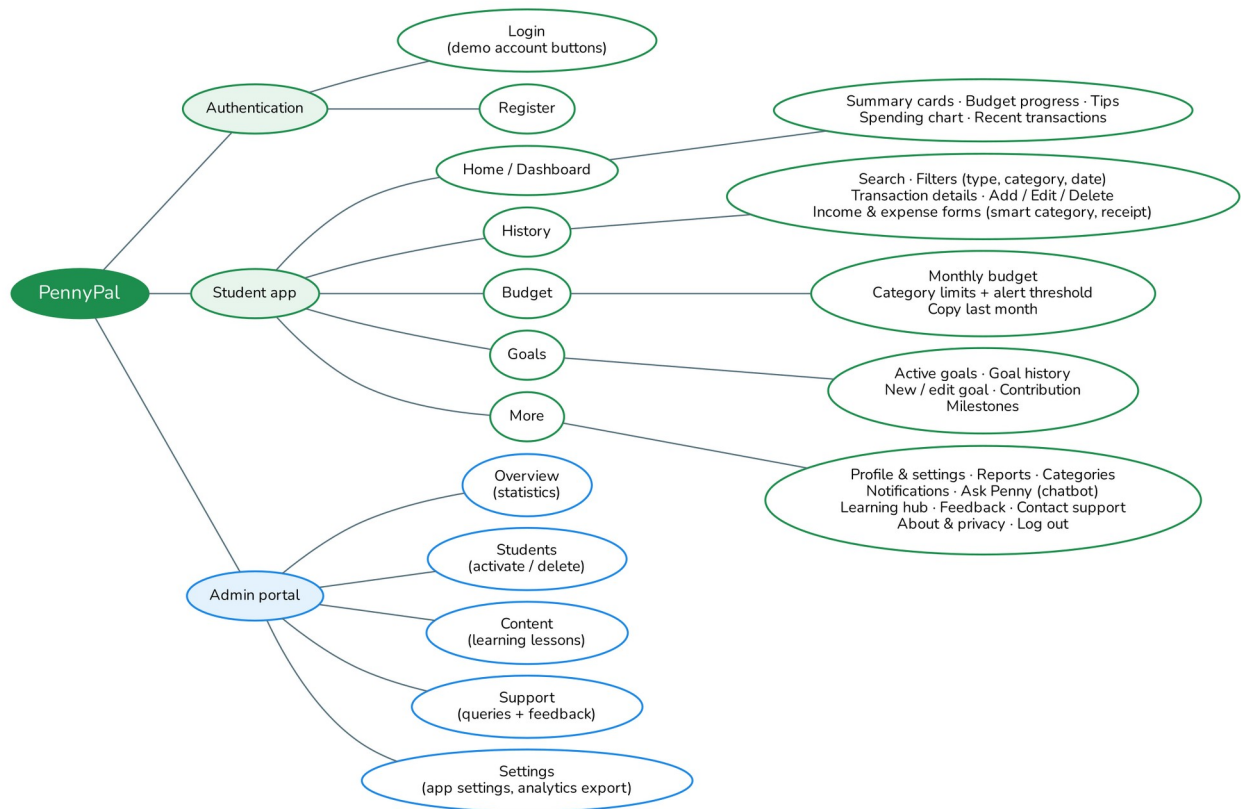


Figure 2: PennyPal site map

2.5 User Interface Design

- **Visual identity:** fresh green brand colour (#1E8E4E) matching the tagline "Fresh All Along", rounded cards, clear icons and the bundled Nunito font so the app looks the same on every platform and works offline.
- **Colour meaning:** green for income and healthy budgets, amber when a budget reaches its alert threshold, red for expenses and overspending.
- **Responsive layout:** bottom navigation on phones; navigation rail and wider grids on tablets and web; content width capped so text stays readable.
- **Beginner-friendly language:** simple labels ("Where your money went this month"), helpful validation messages and friendly empty states that say what to do next.
- **Accessibility:** screen-reader labels on charts, progress bars, icons and logo; tooltips on icon buttons; 48 px touch targets; sufficient colour contrast; light and dark themes follow the system setting.

Screenshots of the main screens are included in Appendix A.

2.6 Key Business Rules and Algorithms

Budget alerts

When an expense is saved, the app calculates, for the expense's month, the spending against the overall budget and against the budget of the expense's category, both before and after the change. An alert is raised only when this particular expense crosses a line: usage moves from below the alert threshold to at or above it ("Approaching your limit"), or from at most 100% to above 100% ("Budget exceeded"). This means the student is warned once instead of on every later purchase. Alerts are

shown immediately, saved to Notifications and sent as a device notification if the student has notifications turned on.

Smart expense categorisation

While the student types a description the categoriser suggests a category in three steps: (1) if a previous expense has a very similar description (similarity 0.72 or higher), reuse its category, so the app learns from the student's own corrections; (2) look for known keywords for each category, such as "bus", "netflix" or "textbook"; (3) otherwise compare each word with the keywords and custom category names using the Dice coefficient, accepting matches of 0.6 or higher, so typos like "piza" still map to Food. The suggestion is only pre-selected until the student picks a category themselves.

Savings goal calculations

- $\text{Progress} = \text{current amount} \div \text{target amount}$; $\text{remaining} = \text{target} - \text{current}$.
- $\text{Months to go} = \text{remaining} \div \text{monthly contribution (rounded up)}$; $\text{estimated completion} = \text{today} + \text{months to go}$.
- A goal is "behind schedule" when the estimated completion is after the target date; the app then shows the monthly amount needed to finish on time.
- Milestones at 25, 50, 75 and 100% are marked automatically (and can be marked manually). At 100% the goal is completed and moved to Goal history.

Personalised tips

The dashboard shows rule-based recommendations: budgets over or near their limit, spending more than income, a strong savings rate (20% or more), a category taking over 35% of spending, spending faster than at the same point last month, no budget set, and goals behind schedule.

Chatbot

The chatbot sends the question, recent conversation and a summary of the month (income, expenses, top categories, budget usage and goals) to Gemini with instructions to give short, educational budgeting advice and to decline investment or loan questions. No names, emails or phone numbers are sent. If no API key is configured, the request fails, or no answer arrives within 10 seconds, the offline rule-based advisor answers using the same summary.

Offline synchronisation

Every create, update or delete writes to SQLite and adds an entry to the SyncQueue table. Older queue entries for the same record are replaced, so repeated edits made offline are sent only once. When the student is signed in to Firebase and the device is online, queued entries for that student are pushed to Firestore using the record's ID as the document ID, which makes retries safe and prevents duplicates. Failed entries are retried later and skipped after five attempts so that one bad record never blocks the others.

2.7 Security and Privacy Design

Table 7: Security and privacy measures

Area	Design decision
Passwords	Never stored in plain text. PBKDF2-HMAC-SHA256 with 10,000 iterations and a random 16-byte salt per user; verification uses a constant-time comparison.
Password policy	At least 8 characters with upper case, lower case, a number and a symbol.
Access control	Every query is filtered by the logged-in user's ID; admin screens are only reachable with the admin role; deactivated accounts cannot log in.
Cloud rules	Firestore rules allow students to read and write only documents they own; only the administrator can change shared content and settings.

Area	Design decision
Transmission	Firebase and Gemini are called over HTTPS. The password hash is removed before any record is uploaded.
Secrets	API keys are passed at build time and are not stored in the source code.
Data minimisation	Only name, email, mobile number and the records the student enters are collected. No bank, card or location data.
Safety	No external links, downloads or redirects; destructive actions (delete, deactivate, log out) always ask for confirmation.

2.8 Non-Functional Requirements

Table 8: Non-functional requirements

Requirement	How PennyPal satisfies it
Safety	No downloads or redirects to external sites; confirmation dialogs before destructive actions.
Accessibility	Semantic labels for charts, images and progress bars, tooltips, large touch targets, readable contrast, system dark mode, text that scales.
User-friendliness	Plain language, clear menus, one-tap demo logins, inline validation messages, empty-state guidance.
Performance	Data kept in memory after the first load; charts and totals computed locally; no network needed for normal use.
Reliability	Input validation on every form, database constraints, try/catch around camera, notifications, sync and AI calls; duplicate-safe sync.
Security	Hashed passwords, per-user access, HTTPS, Firestore rules (see 2.7).
Privacy	Privacy notice shown at registration and in About; chatbot receives only totals.
Scalability	Layered design, UUID keys, custom categories, admin-managed content, and cloud storage allow more users, categories and modules without redesign.
Compatibility	Android 6.0+, iOS, tablets and modern browsers from one codebase.
Maintainability	Code organised by layer and module, commented, analysed with lints and covered by automated tests; the database schema lives in one SQL file used by both the app and the submission.

2.9 Requirements Traceability

Table 9: Traceability to the specification

Requirement (SRS 1.6)	Implemented in
Register with name, email, mobile, password	Register screen
Secure login with email and password; logout	Login screen; More / Profile / Admin settings → Log out
Validate required fields, email, password rules, duplicates	Register form validation; duplicate email check
Dashboard: income, expenses, balance, budget status	Home: balance card and summary tiles
Quick navigation to Add Income/Expense, Budget, History, Learning, Feedback, Support	Home: Quick actions
Visual indicators and empty-data messages	Home: cards, progress bars, donut chart, empty states
Add income (source, amount, date, description)	Add income form

Requirement (SRS 1.6)	Implemented in
Add expense (category, amount, date, description)	Add expense form
View, edit, delete income and expenses	History, transaction details sheet
Default expense categories	Food, Transport, Education, Shopping, Entertainment, Bills, Savings, Miscellaneous
Search and filter by date, category, type	History: search box, type buttons, category and date-range filters
Monthly budget, comparison, remaining, overspending, update/delete	Budget screen
Learning section with topics and readable lessons	Learning hub and lesson screen
About, Feedback (name, email, rating, comments), Support (subject, message), confirmations	About & privacy, Feedback, Contact support
Chatbot for budgeting questions	Ask Penny
Savings goals with target, current, date, monthly contribution	Goal form
Remaining amount, progress %, estimated completion	Goal card
Update contributions, mark milestones	Add contribution dialog; milestone chips
Progress charts, badges, motivational messages	Progress ring, milestone badges, messages on goal card
Completed goals archived in history	Goals → History tab
Instant notifications near thresholds	Budget monitor alerts and notifications
Camera images on transactions (optional)	Attach receipt photo
AI categorisation (optional)	Smart category suggestion
Reports and spending trends	Spending reports
Admin: manage users, monitor activity, analytics, content	Admin portal

3. Diagrams

3.1 Data Flow Diagram – Level 0 (Context)

The context diagram shows PennyPal as a single process and the external entities that exchange data with it.

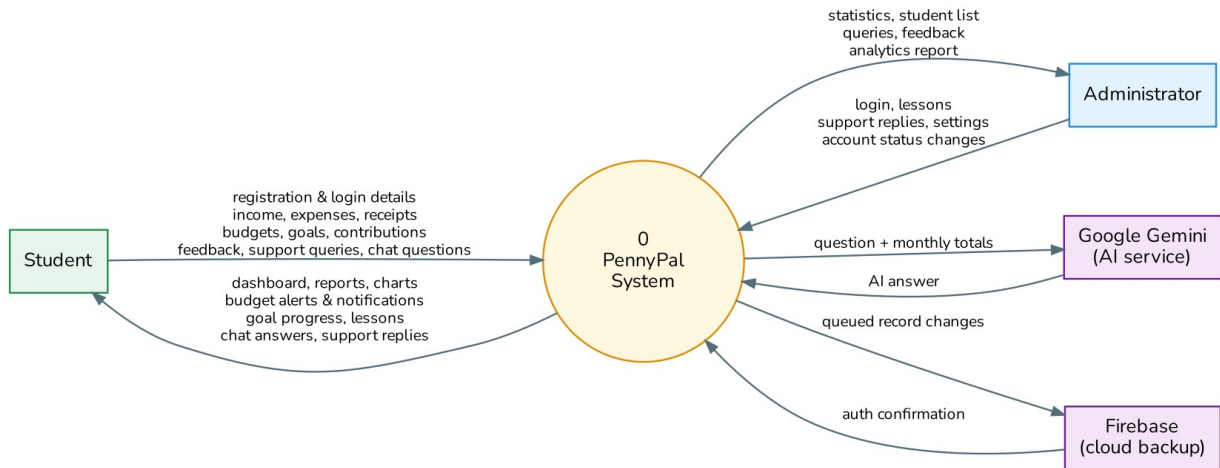


Figure 3: DFD Level 0

3.2 Data Flow Diagram – Level 1

Level 1 breaks the system into nine processes and shows the data stores (D1–D9) each one reads or writes.

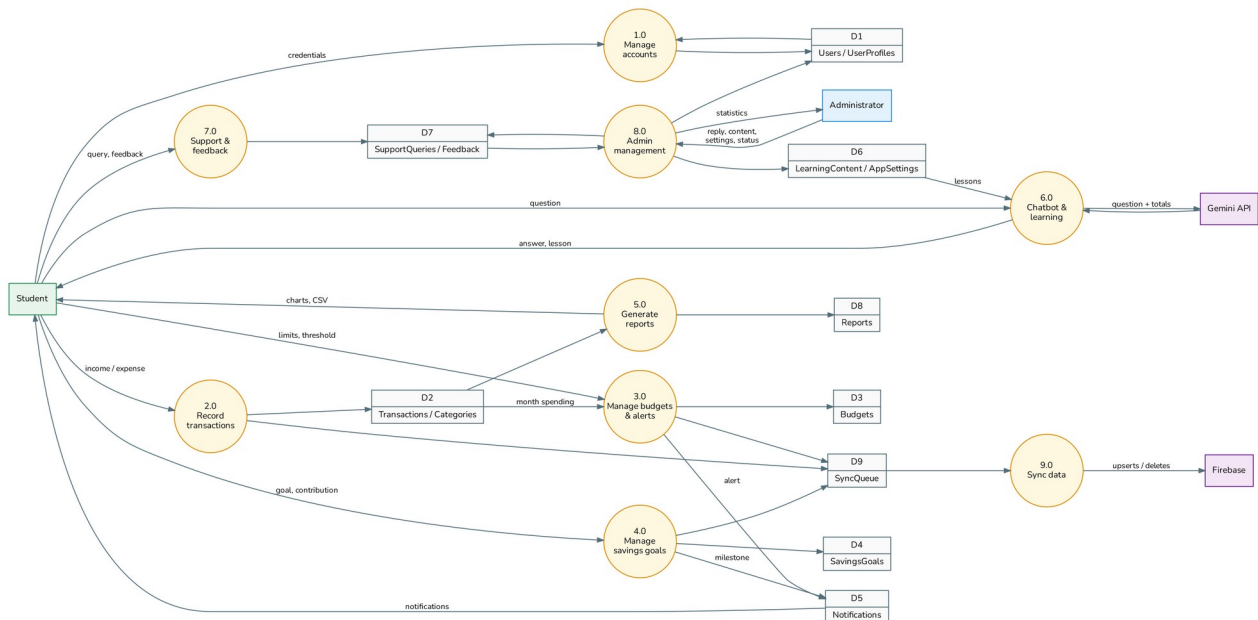


Figure 4: DFD Level 1

3.3 Flowcharts

Legend: green ovals are start/end points, white boxes are processing steps, blue parallelograms are user input or output, and amber diamonds are decisions.

3.3.1 Registration and login

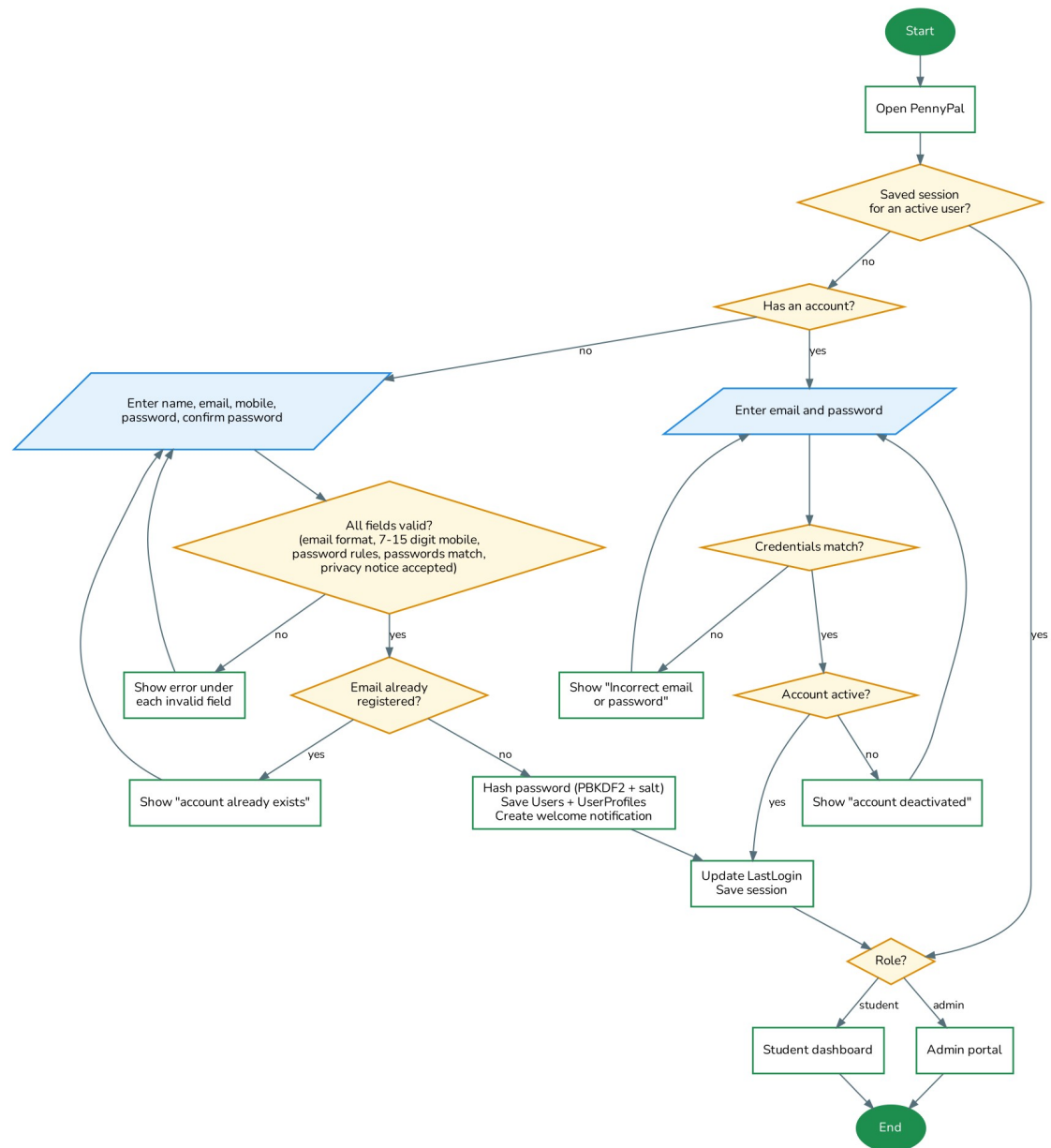


Figure 5: Registration and login flow

3.3.2 Adding an expense with real-time budget alert

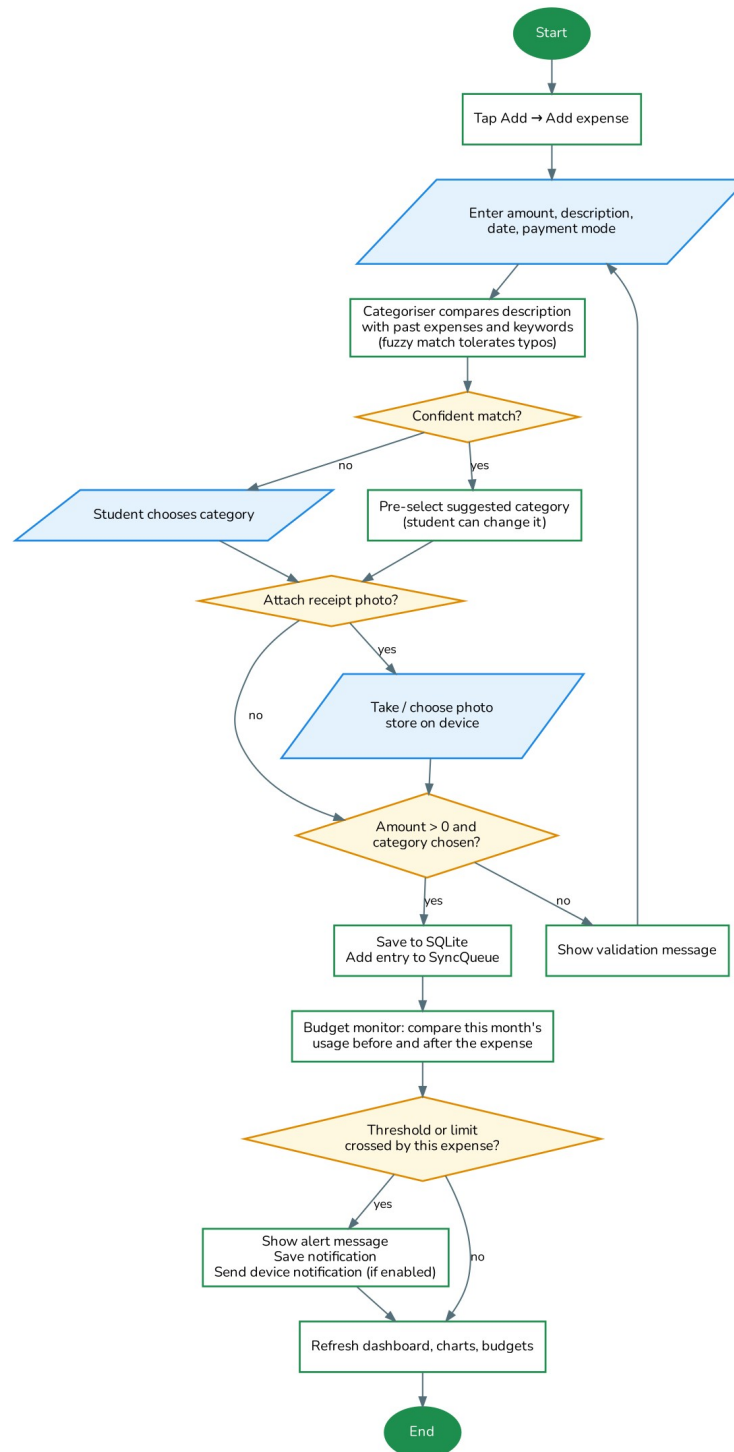


Figure 6: Add expense and budget alert flow

3.3.3 Savings goal contribution

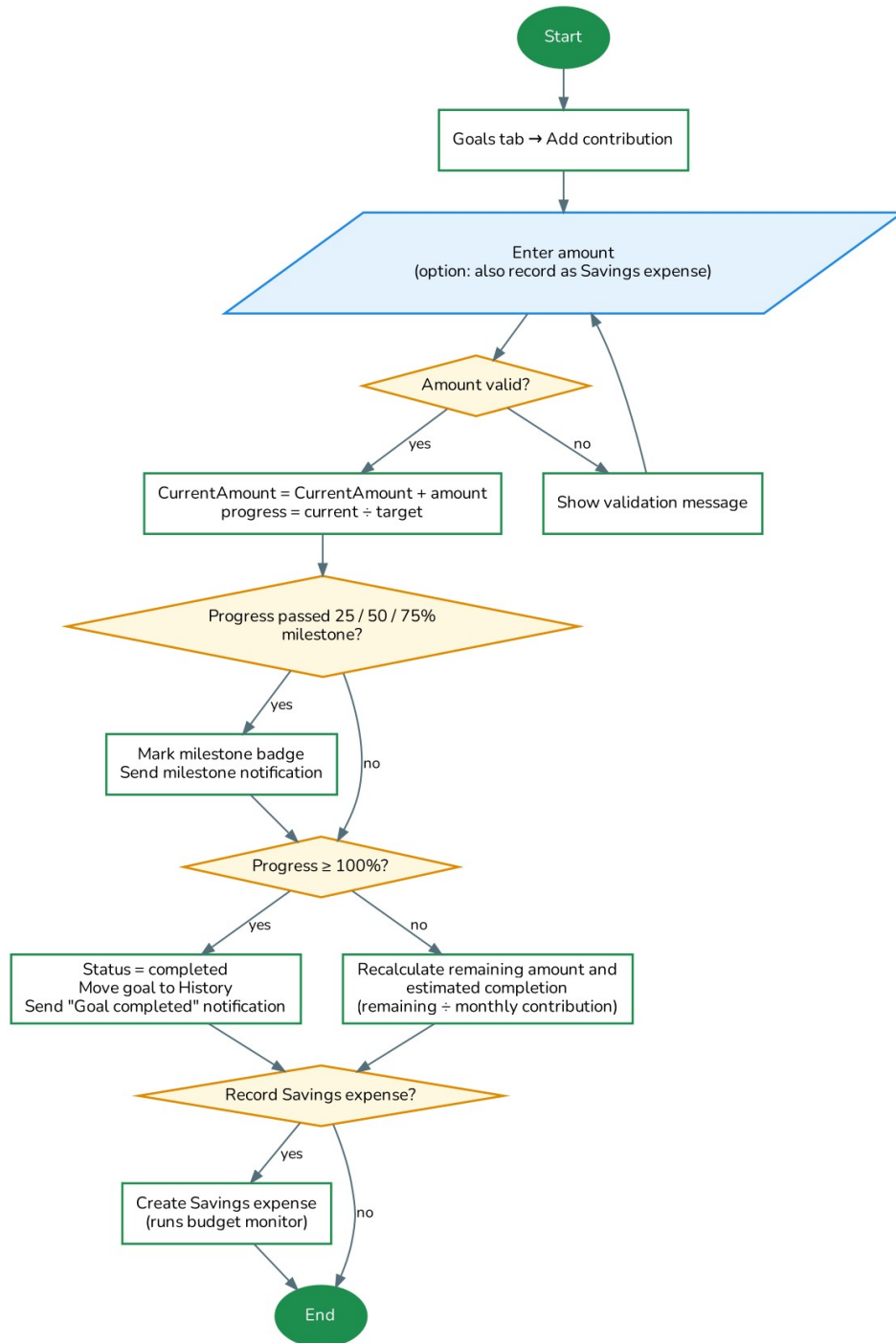


Figure 7: Savings contribution flow

3.3.4 Chatbot question

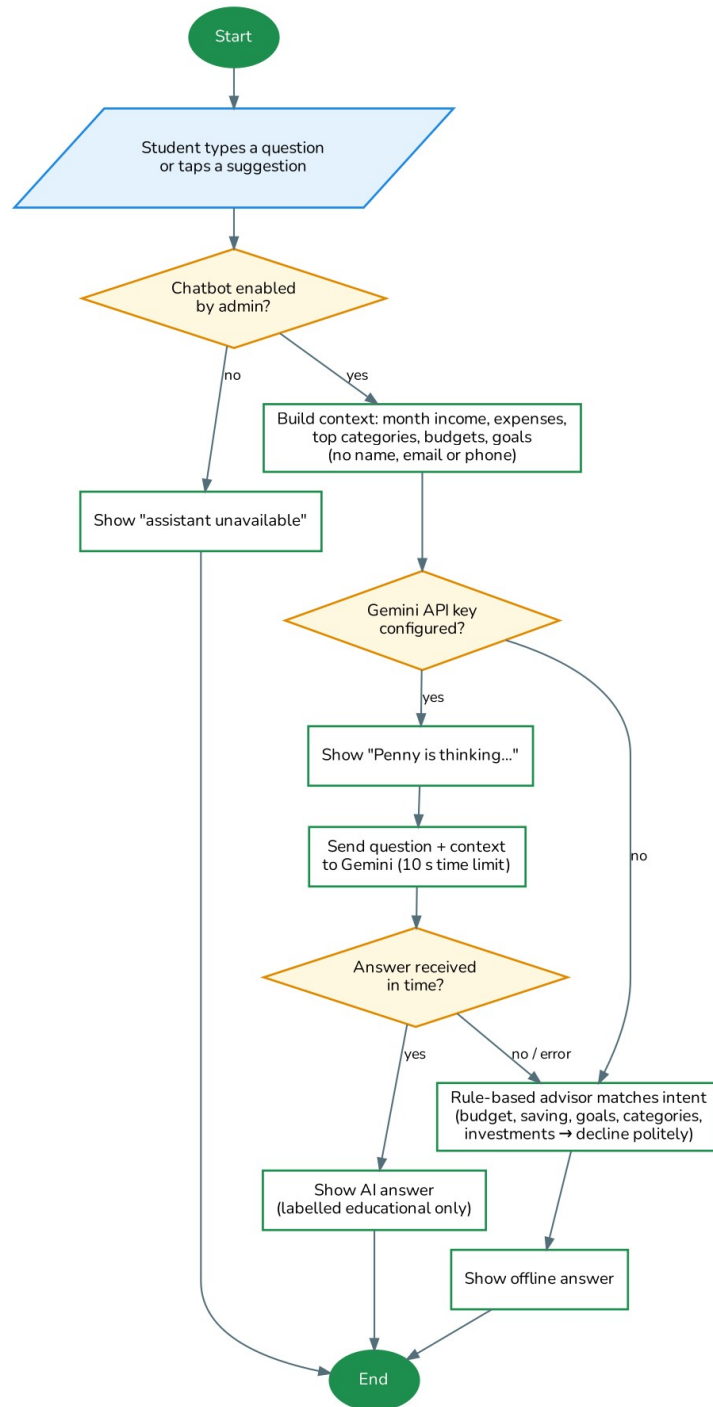


Figure 8: Chatbot flow with offline fallback

3.3.5 Offline synchronisation

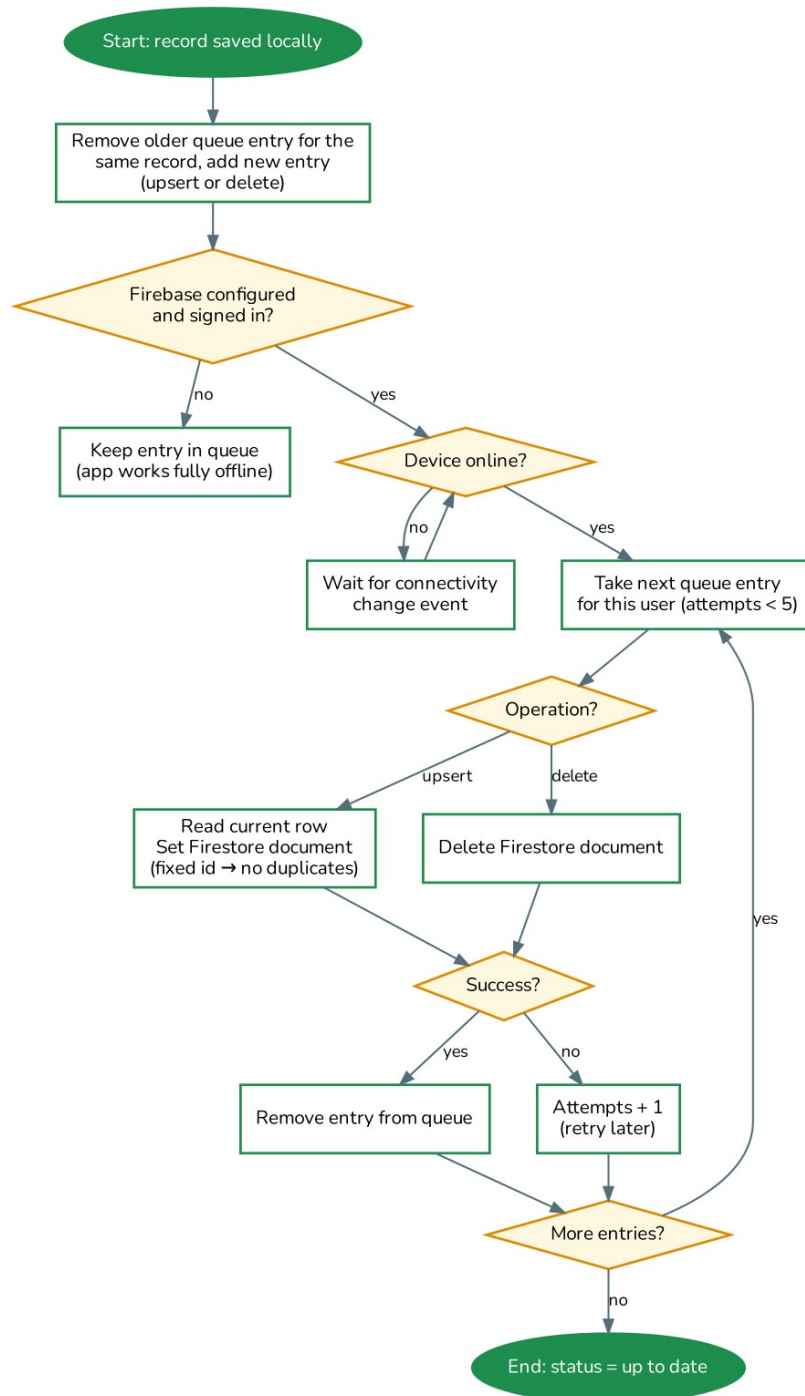


Figure 9: Offline sync flow

3.3.6 Administrator tasks

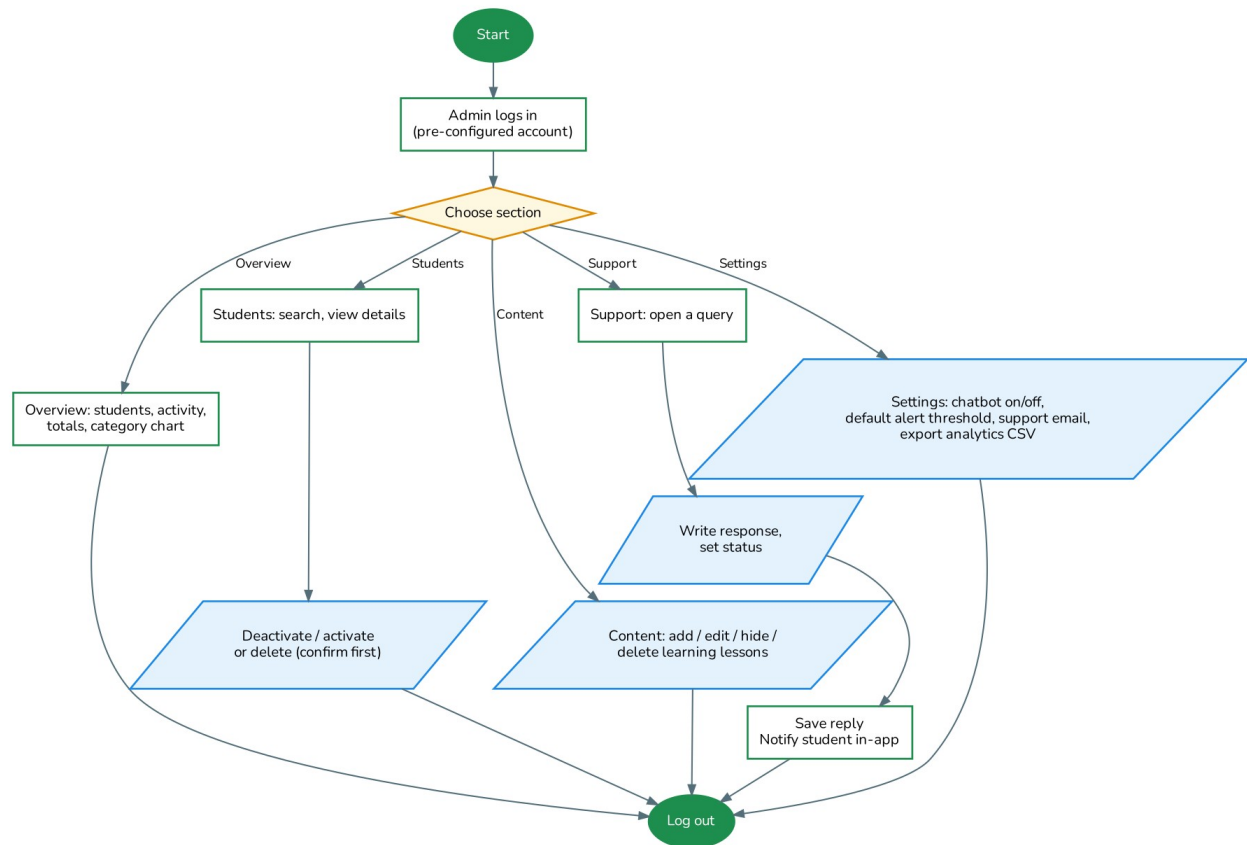


Figure 10: Administrator flow

3.4 Entity Relationship Diagram

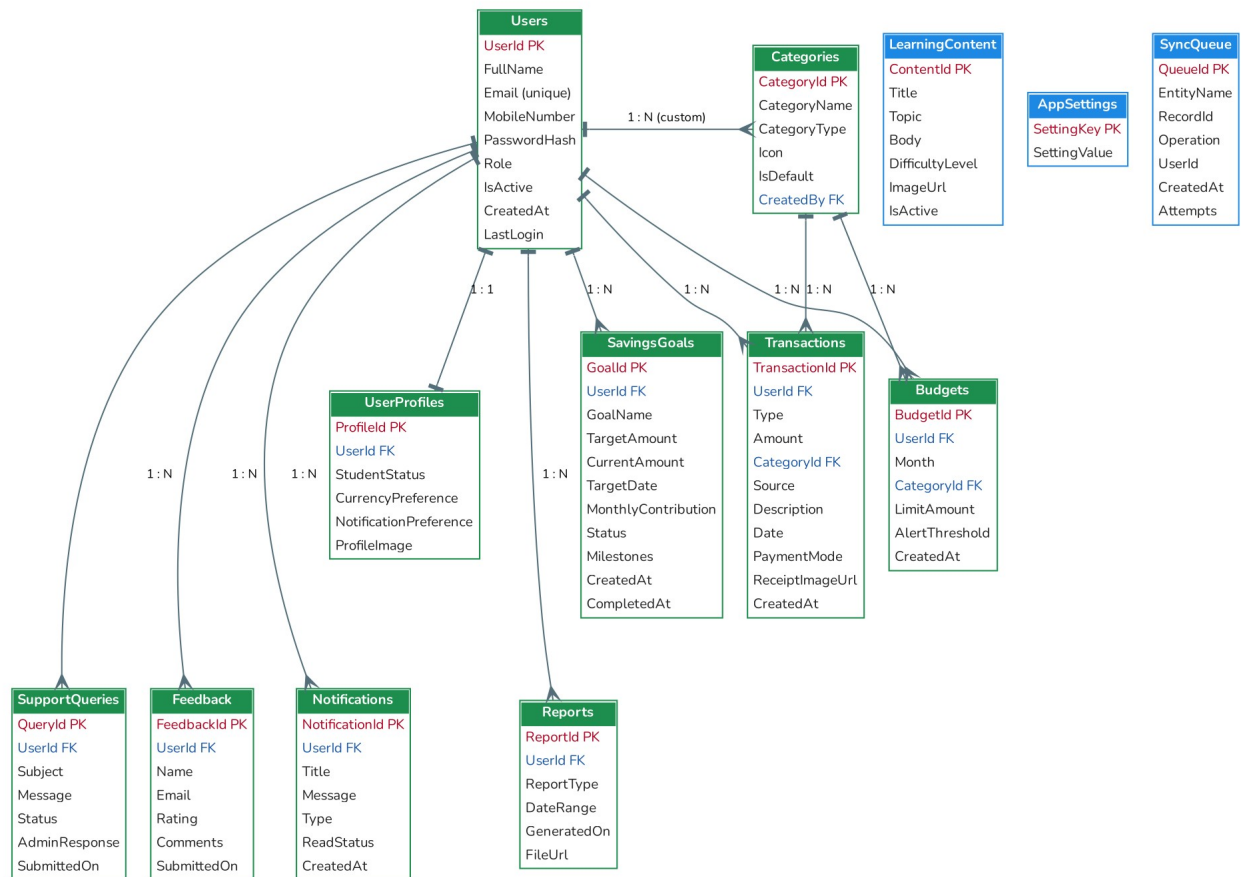


Figure 11: Entity relationship diagram (PK = primary key, FK = foreign key)

4. Database Design

4.1 Overview

- The on-device database is SQLite. It is created on first launch from the same script that is submitted (pennypal_schema.sql), so the documented design and the running app always match.
- Primary keys are text UUIDs, generated on the device, so records created offline on different devices never collide when synced.
- Dates are stored as ISO-8601 text (YYYY-MM-DD for transaction dates, full timestamps elsewhere); months are stored as YYYY-MM.
- Foreign keys are enforced. Deleting a user removes that user's profile, transactions, budgets, goals, reports, notifications and support queries (ON DELETE CASCADE).
- When cloud sync is enabled, the same records are mirrored in Cloud Firestore (see 4.4).

4.2 Table Definitions

Users

One row per account (students and the administrator).

Column	Type	Constraints	Description
UserId	TEXT	PK	UUID of the user
FullName	TEXT	NOT NULL	Name entered at registration
Email	TEXT	NOT NULL, UNIQUE (case-insensitive)	Login ID
MobileNumber	TEXT	NOT NULL	7–15 digits, optional +
PasswordHash	TEXT	NOT NULL	PBKDF2 hash with salt
Role	TEXT	student admin	Access level
IsActive	INTEGER	Default 1	0 = deactivated by admin
CreatedAt	TEXT	NOT NULL	Registration time
LastLogin	TEXT		Last successful login

UserProfiles

Preferences for each user (one-to-one with Users).

Column	Type	Constraints	Description
ProfileId	TEXT	PK	UUID
UserId	TEXT	FK → Users, UNIQUE	Owner
StudentStatus	TEXT		e.g. Undergraduate
CurrencyPreference	TEXT	Default USD	Display currency
NotificationPreference	INTEGER	Default 1	Device notifications on/off
ProfileImage	TEXT		Reserved for a profile picture

Categories

Expense categories: 8 defaults plus custom categories created by students.

Column	Type	Constraints	Description
CategoryId	TEXT	PK	e.g. cat-food
CategoryName	TEXT	NOT NULL	Display name
CategoryType	TEXT	expense income	Kind of category
Icon	TEXT		Icon/colour key
IsDefault	INTEGER	Default 0	1 for built-in categories
CreatedBy	TEXT	FK → Users	Owner of a custom category

Transactions

Income and expense records.

Column	Type	Constraints	Description
TransactionId	TEXT	PK	UUID
UserId	TEXT	FK → Users	Owner
Type	TEXT	income expense	
Amount	REAL	NOT NULL, > 0	Amount in the student's currency
CategoryId	TEXT	FK → Categories	Expenses only
Source	TEXT		Income only, e.g. Monthly allowance
Description	TEXT		Free text (max 120 characters in the app)
Date	TEXT	NOT NULL	YYYY-MM-DD
PaymentMode	TEXT		Cash, Card, Bank transfer, Mobile money, ...
ReceiptImageUrl	TEXT		File path (mobile) or embedded image (web)
CreatedAt	TEXT	NOT NULL	Entry time

Budgets

Monthly overall budget (CategoryId empty) and category limits. A unique index allows one budget per user, month and category.

Column	Type	Constraints	Description
BudgetId	TEXT	PK	UUID
UserId	TEXT	FK → Users	Owner
Month	TEXT	NOT NULL	YYYY-MM
CategoryId	TEXT	FK → Categories, nullable	Empty = overall monthly budget
LimitAmount	REAL	NOT NULL, > 0	Budget or limit
AlertThreshold	INTEGER	1–100, default 80	Percentage that triggers a warning
CreatedAt	TEXT	NOT NULL	

SavingsGoals

Savings goals and their progress.

Column	Type	Constraints	Description
GoalId	TEXT	PK	UUID
UserId	TEXT	FK → Users	Owner
GoalName	TEXT	NOT NULL	e.g. New laptop
TargetAmount	REAL	NOT NULL, > 0	
CurrentAmount	REAL	Default 0	Saved so far
TargetDate	TEXT	NOT NULL	YYYY-MM-DD
MonthlyContribution	REAL	Default 0	Planned monthly saving
Status	TEXT	active completed	Completed goals appear in history
Milestones	TEXT		Reached percentages, e.g. "25,50"
CreatedAt	TEXT	NOT NULL	
CompletedAt	TEXT		Completion time

Reports

History of generated/exported reports.

Column	Type	Constraints	Description
ReportId	TEXT	PK	UUID
UserId	TEXT	FK → Users	Owner
ReportType	TEXT	NOT NULL	monthly custom
DateRange	TEXT	NOT NULL	"from..to"
GeneratedOn	TEXT	NOT NULL	
FileUrl	TEXT		Exported file name

LearningContent

Lessons shown in the learning hub, managed by the administrator.

Column	Type	Constraints	Description
ContentId	TEXT	PK	
Title	TEXT	NOT NULL	
Topic	TEXT	NOT NULL	Budgeting, Saving, Income, Expenses, Requirements
Body	TEXT	NOT NULL	Lesson text
DifficultyLevel	TEXT	Default Beginner	
ImageUrl	TEXT		Illustration key
IsActive	INTEGER	Default 1	0 = hidden from students

Notifications

In-app notifications (budget alerts, goal milestones, support replies, welcome message).

Column	Type	Constraints	Description
NotificationId	TEXT	PK	
UserId	TEXT	FK → Users	Recipient
Title	TEXT	NOT NULL	
Message	TEXT	NOT NULL	
Type	TEXT	NOT NULL	budget goal system
ReadStatus	INTEGER	Default 0	1 = read
CreatedAt	TEXT	NOT NULL	

SupportQueries

Contact-support requests and admin replies.

Column	Type	Constraints	Description
QueryId	TEXT	PK	
UserId	TEXT	FK → Users	Student
Subject	TEXT	NOT NULL	
Message	TEXT	NOT NULL	
Status	TEXT	open in_progress resolved	
AdminResponse	TEXT		Reply shown to the student
SubmittedOn	TEXT	NOT NULL	

Feedback (added)

Feedback form submissions. Added to the reference design because the Feedback screen stores name, email, rating and comments.

Column	Type	Constraints	Description
FeedbackId	TEXT	PK	
UserId	TEXT	FK → Users, nullable	Student who sent it
Name	TEXT	NOT NULL	
Email	TEXT	NOT NULL	
Rating	INTEGER	1–5	Star rating
Comments	TEXT	NOT NULL	
SubmittedOn	TEXT	NOT NULL	

AppSettings (added)

Key/value settings changed by the administrator (chatbot_enabled, default_alert_threshold, support_email).

Column	Type	Constraints	Description
SettingKey	TEXT	PK	Setting name

Column	Type	Constraints	Description
SettingValue	TEXT	NOT NULL	Setting value

SyncQueue (added)

Local changes waiting to be uploaded to the cloud.

Column	Type	Constraints	Description
QueueId	INTEGER	PK, auto-increment	Order of changes
EntityName	TEXT	NOT NULL	Table name
RecordId	TEXT	NOT NULL	Changed record
Operation	TEXT	upsert delete	
UserId	TEXT		Owner, used to sync only the signed-in user's changes
CreatedAt	TEXT	NOT NULL	
Attempts	INTEGER	Default 0	Failed upload attempts

4.3 Relationships

Table 10: Relationships and integrity rules

Relationship	Type	Rule on delete
Users → UserProfiles	1 : 1	Cascade
Users → Transactions, Budgets, SavingsGoals, Reports, Notifications, SupportQueries	1 : N	Cascade
Users → Feedback	1 : N	Set null (feedback is kept)
Users → Categories (custom)	1 : N	Set null
Categories → Transactions	1 : N	Deleting a custom category moves its expenses to Miscellaneous
Categories → Budgets	1 : N	Budgets for a deleted custom category are removed

4.4 Cloud Firestore Structure (optional)

- students/{UserId}/{TableName}/{RecordId} – each student's records, with an ownerId field linking them to the Firebase account.
- LearningContent/{ContentId} and AppSettings/{SettingKey} – shared content, readable by signed-in users and writable only by the administrator.
- Security rules are provided in firestore.rules.

4.5 SQL Scripts Submitted

Table 11: Database scripts

File	Contents
database/pennypal_schema.sql	CREATE TABLE and index statements for all 13 tables.

File	Contents
database/pennypal_seed_data.sql	INSERT statements for the seed and test data (admin account, demo student, categories, 38 transactions, budgets, goals, lessons, settings, sample support query, feedback and notification).

5. Test Data and Testing

5.1 Test Data

On first launch the app loads the following data. The same data is provided in pennypal_seed_data.sql. Demo transactions are dated relative to the current month so the dashboard always has data; the figures below are for the current month at the time of testing (September 2026).

Table 12: Seed and test data

Data set	Records
Accounts	Administrator (admin@pennypal.app) and demo student Ada Student (student@pennypal.app, Undergraduate, USD)
Categories	Food, Transport, Education, Shopping, Entertainment, Bills, Savings, Miscellaneous
Transactions	38 records over three months: 11 expenses per month plus a monthly allowance of \$500.00 and part-time tutoring income of \$180.00 (in two of the three months)
Budgets (current month)	Overall \$450.00; Food \$100.00; Entertainment \$30.00; Shopping \$60.00 (all alert at 80%)
Savings goals	New laptop \$320.00 of \$900.00 (\$75.00/month); Emergency fund \$150.00 of \$500.00 (\$50.00/month); Concert ticket \$120.00 of \$120.00 (completed)
Learning content	9 lessons across Budgeting, Saving, Income, Expenses and Requirements
Engagement	1 resolved support query, 1 five-star feedback, 1 welcome notification
App settings	Chatbot enabled; default alert threshold 80%; support email support@pennypal.app

Table 13: Monthly demo transactions (amounts shown as current / last / two months ago where they differ)

Day	Description	Category	Payment	Amount
1	Monthly allowance	Income	Bank transfer	+500.00
2	Groceries at supermarket	Food	Card	42.50
3	Monthly bus pass	Transport	Card	25.00
5	Netflix subscription	Entertainment	Card	12.99
7	Pizza with friends	Food	Cash	18.40
9	Phone data plan	Bills	Mobile money	35.00
11	Textbooks and printing	Education	Card	60.00 / 75.00 / 90.00
13	Cafeteria lunches	Food	Cash	22.75
15	Part-time tutoring	Income	Bank transfer	+180.00
16	New t-shirt	Shopping	Card	45.00 / 40.00 / 35.00
20	Transfer to savings	Savings	Cash	50.00
22	Uber ride home	Transport	Card	9.50
25	Cinema tickets	Entertainment	Cash	15.00

Expected current-month totals with this data: income \$680.00, expenses \$336.14, overall budget 75% used (\$113.86 left), Entertainment 93% used and Food 84% used (both above the 80% alert threshold).

5.2 Test Cases

The test cases below were run on the web build of the app (phone and desktop screen sizes) and through the automated tests. Items that depend on phone hardware are marked for verification on a physical device.

Table 14: Test cases

ID	Module	Test / input	Expected result	Result
TC01	Login	Demo student: student@pennypal.app / Student@123	Student dashboard opens	Pass
TC02	Login	admin@pennypal.app / wrong password	"Incorrect email or password."	Pass
TC03	Login	Admin: admin@pennypal.app / Admin@123	Admin portal opens	Pass
TC04	Register	Name "A", email "not-an-email", mobile "12", empty password	Error under each field: name too short, invalid email, 7–15 digits, password required	Pass
TC05	Register	Password "alldlowercase1!"	Message asks for an upper-case letter	Pass
TC06	Register	Valid details with an email that already exists (different case)	"An account with this email already exists."	Pass
TC07	Register	Grace Hopper, grace@uni.edu, +1 555 010 0300, Compiler#1	Account created, mobile saved as +15550100300, welcome notification	Pass
TC08	Login	Student deactivated by admin tries to log in	"This account has been deactivated..."	Pass
TC09	Dashboard	Log in as demo student	Income \$1,860.00 total (\$680.00 this month), expenses \$1,038.42 (\$336.14), balance \$821.58, budget 75% used	Pass
TC10	Dashboard	Register Grace Hopper (grace@uni.edu, +15550100300, Compiler#1) and open the dashboard	All totals \$0.00, budget "Not set", "No budget for this month" and "No expenses this month" messages, "Start tracking" tip	Pass
TC11	Expense	Amount 20, description "piza with friends"	Smart category suggests Food (similar to "Pizza with friends")	Pass
TC12	Budget alert	Save TC11 expense (Food at \$83.65 of \$100)	"Budget exceeded ... Food budget ... by \$3.65" message and notification	Pass
TC13	Budget alert	Food at \$70 of \$100, add \$15	One "Approaching your limit" alert; a further \$5 expense gives no repeat alert	Pass
TC14	Expense	Amount 0 or "abc"	"Amount must be greater than zero" / "Enter a number"	Pass
TC15	Expense	Edit "Uber ride home" from 9.50 to 12; then delete it and confirm	"Changes saved" and amount shows –\$12.00; after delete the record disappears and totals update	Pass
TC16	History	Search "uber"	3 results, total –\$28.50	Pass
TC17	Budget	Save Transport limit twice for the same month	Second save updates the first (no duplicate)	Pass
TC18	Budget	Open next month (no budget) → Copy last month	"Copied 4 budget(s) from last month"; overall \$450 and 3 category limits at \$0.00 used	Pass

ID	Module	Test / input	Expected result	Result
TC19	Goals	Contribute remaining \$350.00 to Emergency fund	Milestones 50/75/100% reached, goal completed and moved to History	Pass
TC20	Goals	Goal \$1,000, saved \$400, \$100/month	Remaining \$600, 40%, 6 months to go	Pass
TC21	Categories	Add custom "Gym", add expense, delete "Gym"	Expense moves to Miscellaneous	Pass
TC22	Reports	This month	Income \$680.00, expenses \$336.14, net \$343.86, donut chart and 6-month bars	Pass
TC23	Reports	Export CSV	CSV file shared/downloaded and listed under Generated reports	Verify on device
TC24	Chatbot	"How am I doing this month?" (no API key)	Offline answer with income, spending and budget usage	Pass
TC25	Chatbot	"Should I buy bitcoin?"	Polite refusal recommending a qualified adviser	Pass
TC26	Chatbot	With a Gemini key	AI answer labelled "AI-generated · educational only" within 10 s	Verify on device
TC27	Support	Submit subject + message; admin replies "Use Reports > Export."	Student sees reply and a "Support replied" notification	Pass
TC28	Feedback	Comment "Budget alerts are really helpful." without a rating, then with 4 stars	"Please choose a rating"; then "Thank you!" confirmation dialog	Pass
TC29	Admin	Delete demo student (confirm)	Student and all their records removed	Pass
TC30	Admin	Hide "The 50/30/20 rule", log out, log in as the student, open Learning hub	Lesson marked hidden in admin list and not shown to the student	Pass
TC31	Logout	More → Log out → confirm; log in as admin	Returns to login; admin portal opens with no student data	Pass
TC32	Offline	Edit the same record three times offline	Only one queue entry kept for that record	Pass
TC33	Receipt	Attach photo with the camera on an Android phone	Photo shown on the form and in transaction details	Verify on device
TC34	Notifications	Cross a budget threshold on an Android phone	System notification appears	Verify on device
TC35	Layout	Open on a 1280 px wide screen	Navigation rail and 4-column summary tiles	Pass

5.3 Automated Tests

The project includes 33 automated tests that run on every push (GitHub Actions) together with static analysis. They use an in-memory copy of the real database schema.

Table 15: Automated test coverage

Group	What is checked
Validators	Email, password rules, mobile number, amount parsing
Password hashing	Correct password accepted, wrong password rejected, random salt

Group	What is checked
Categoriser	Keyword match, typo tolerance, learning from history, no suggestion for gibberish
Analytics	Monthly totals, category totals, overall and category budget usage
Budget monitor	Single alert on crossing threshold, alert on exceeding limit, other categories ignored
Savings goals	Remaining, progress, months to go, behind-schedule logic; contributions complete goals
Offline advisor	Answers with student figures, declines investment questions
Database	Seeded accounts, registration and duplicates, deactivation, unique budgets, sync-queue de-duplication, category deletion, admin statistics, support replies, cascade delete

6. Project Installation Instructions

6.1 Prerequisites

- Flutter SDK 3.27.4 with Dart 3.6 (<https://docs.flutter.dev/get-started/install>)
- Android Studio with Android SDK API 35, an Android emulator or an Android phone running Android 6.0 or later
- JDK 17
- For iOS: a Mac with Xcode 15 or later and CocoaPods
- Development PC: Intel Core i5/i7 or equivalent, 8 GB RAM or more, internet connection for the first build

6.2 Install the APK on an Android phone

1. Copy PennyPal.apk (from the submission, or from the "PennyPal-apk" artifact of the GitHub Actions build) to the phone.
2. Open the file on the phone. If asked, allow installing apps from this source.
3. Tap Install, then Open. The app creates its database and demo data on first launch.
4. Log in with one of the accounts in section 7.

6.3 Run from the source code

1. Unzip the source code (or clone the repository) and open the folder in Android Studio or VS Code.
2. In a terminal in the project folder run: `flutter pub get`
3. Start an emulator or connect a phone with USB debugging enabled, then run: `flutter run`
4. To run in a browser: `flutter run -d chrome`
5. To build the APK: `flutter build apk --release` (output: `build/app/outputs/flutter-apk/app-release.apk`)
6. To build the web version: `flutter build web --release --no-web-resources-cdn` (output: `build/web`)

6.4 Optional: enable the Gemini chatbot

1. Create an API key in Google AI Studio (<https://aistudio.google.com/app/apikey>).
2. Run or build with the key, for example: `flutter run --dart-define=GEMINI_API_KEY=<your key>`
3. Without a key the chatbot uses its built-in offline answers.

6.5 Optional: enable Firebase cloud sync

1. Create a Firebase project; enable Email/Password sign-in and Cloud Firestore.
2. Add a Web app in the Firebase console and note its API key, App ID, Project ID and Sender ID.
3. Publish the security rules from `firestore.rules` (Firebase console → Firestore → Rules, or `firebase deploy --only firestore:rules`).
4. Build with `--dart-define=Firebase_API_KEY=..., Firebase_APP_ID=..., Firebase_PROJECT_ID=... and Firebase_SENDER_ID=...`

6.6 Load the database scripts (optional)

The app creates its database automatically. To inspect the design separately, open a terminal in the database folder and run the two scripts with the SQLite command-line tool: first `pennypal_schema.sql`, then `pennypal_seed_data.sql`, for example: `sqlite3 pennypal.db < pennypal_schema.sql`, followed by `sqlite3 pennypal.db < pennypal_seed_data.sql`

7. User Credentials

Table 16: Login credentials

Role	Login ID (email)	Password	Notes
Administrator	admin@pennypal.app	Admin@123	Pre-configured; cannot be created by registration
Student (demo)	student@pennypal.app	Student@123	Has three months of sample data

New student accounts can be created from the login screen using "Create an account". The login screen also has "Student" and "Administrator" buttons that fill in the demo credentials.

8. Assumptions

- Each device keeps its own local database. Without Firebase, the admin portal manages the accounts and data stored on that device; with Firebase, data is also backed up to the cloud.
- Each student uses one currency, chosen in Profile & settings. There is no currency conversion.
- Available balance = all income recorded – all expenses recorded. Savings contributions can optionally be recorded as a Savings expense.
- Budgets are per calendar month: one overall budget and at most one limit per category per month.
- New budgets start with the administrator's default alert threshold (80%), which the student can change per budget.
- A goal is completed when the saved amount reaches the target; it re-opens if the target is later increased.
- Receipt photos stay on the device (embedded in the record on the web) and are not uploaded to the cloud.
- Feedback, AppSettings and SyncQueue tables were added to the reference database design to support the Feedback screen, admin settings and offline sync.
- The administrator account is pre-configured and cannot be registered through the app.

9. Acknowledgements

9.1 Third-party libraries and resources

Table 17: Libraries and resources

Resource	Used for
Flutter and Dart (Google)	Cross-platform framework
provider	State management
sqlite, sqlite_common_ffl_web	SQLite on mobile and web
fl_chart	Charts
intl	Currency and date formatting
http	Gemini API calls
crypto	Password hashing
uuid	Record IDs
string_similarity	Smart categorisation
image_picker	Receipt photos
share_plus	CSV export
connectivity_plus	Online/offline detection
path_provider, shared_preferences	File locations and saved session
flutter_local_notifications	Device notifications
firebase_core, firebase_auth, cloud_firestore	Optional cloud sync
Google Gemini API / Google AI Studio	Chatbot answers (when configured)
Nunito font (SIL Open Font License)	App typography
Graphviz	Diagrams in this document

9.2 AI tools used

Claude Code (Anthropic) was used as an AI coding assistant during development and to help prepare this documentation. No AI-generated images are used in the app; the app icon is drawn by a script in the project (tool/generate_icons.py).

Appendix A. Screenshots

A.1 Student app

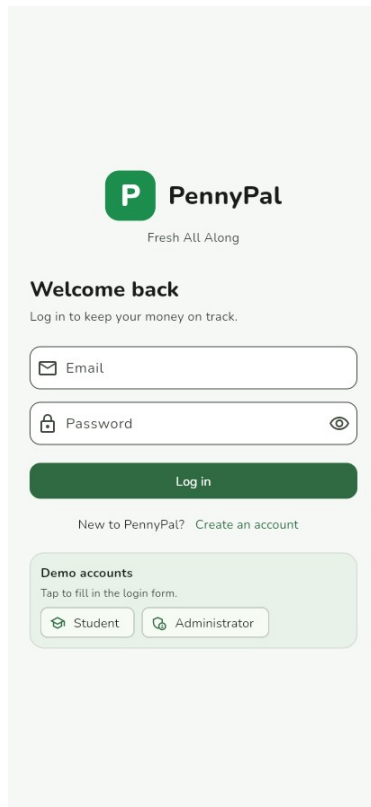


Figure 12: Login

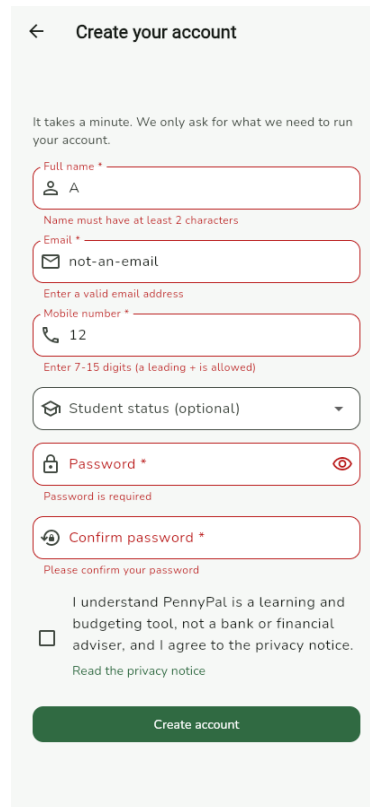


Figure 13: Registration validation

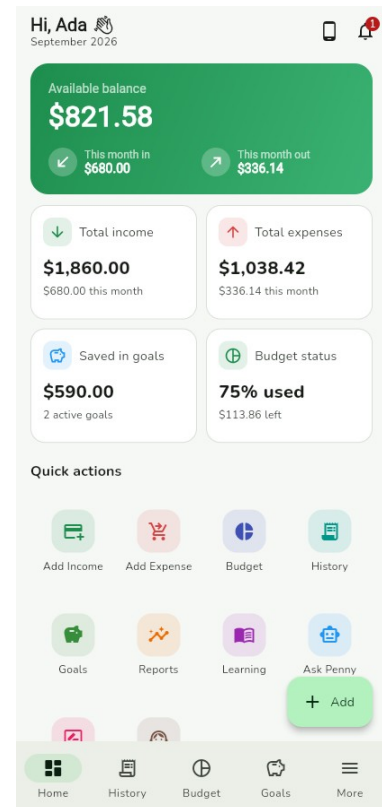


Figure 14: Dashboard

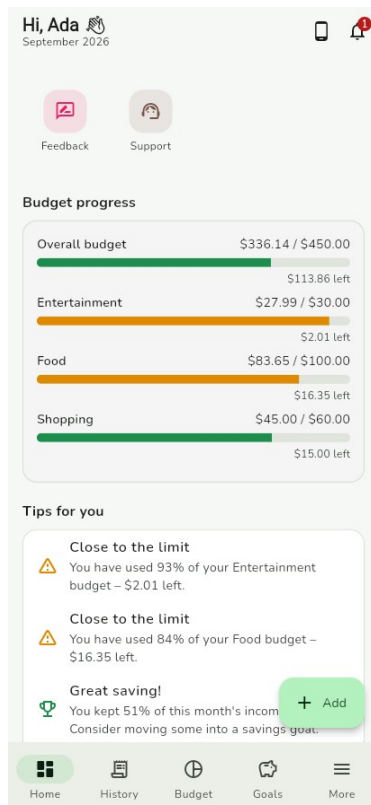


Figure 15: Budget progress and tips

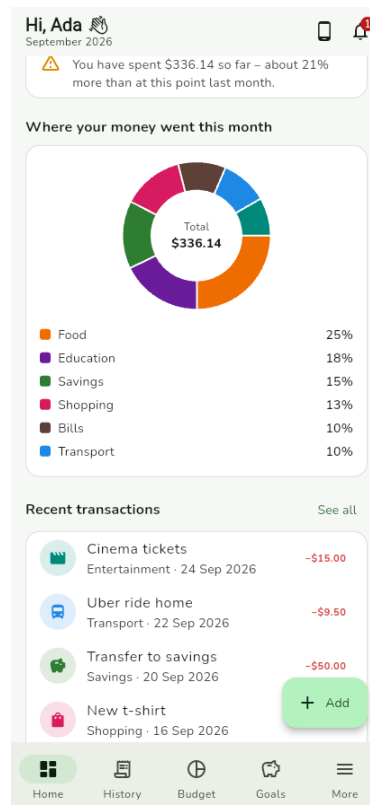


Figure 16: Spending chart

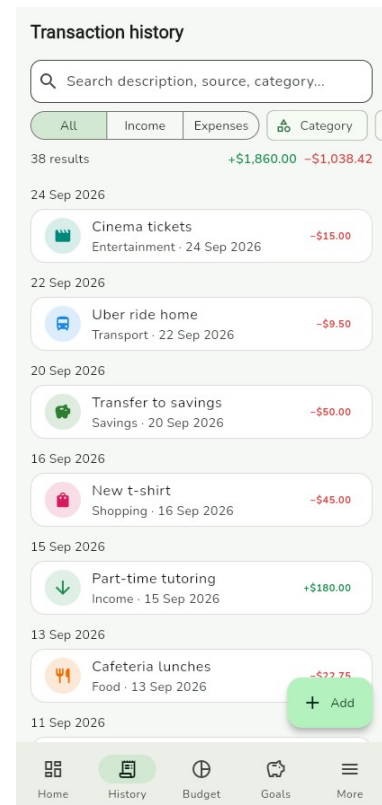


Figure 17: Transaction history

Add expense

Amount *
\$20

What was it for?
pizza with friends

Smart category: Food (Similar to "Pizza with friends")
Category *
Food

Date *
24 Sep 2026

Payment mode
Cash

Receipt (optional)
Attach receipt photo

Save

Figure 18: Add expense with smart category

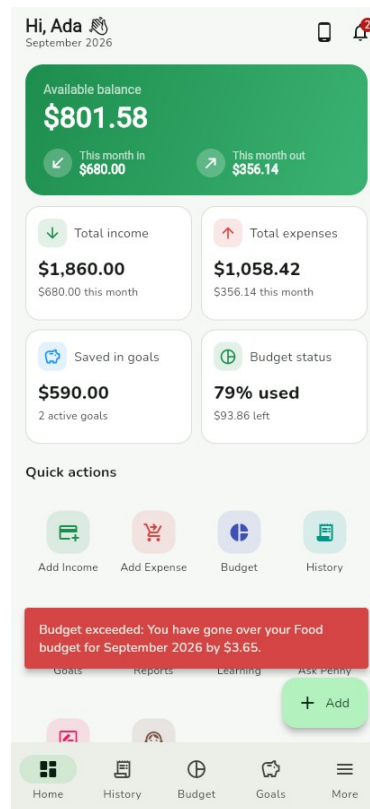


Figure 19: Real-time budget alert

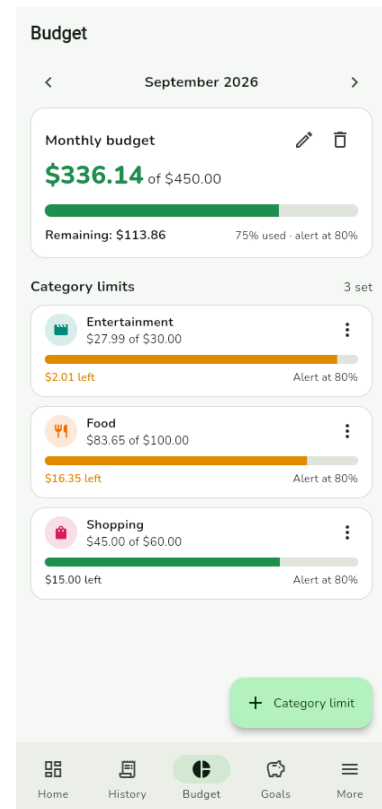


Figure 20: Budget

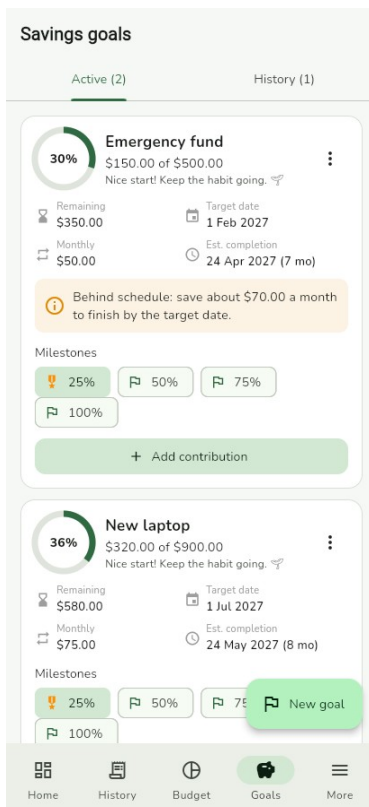


Figure 21: Savings goals

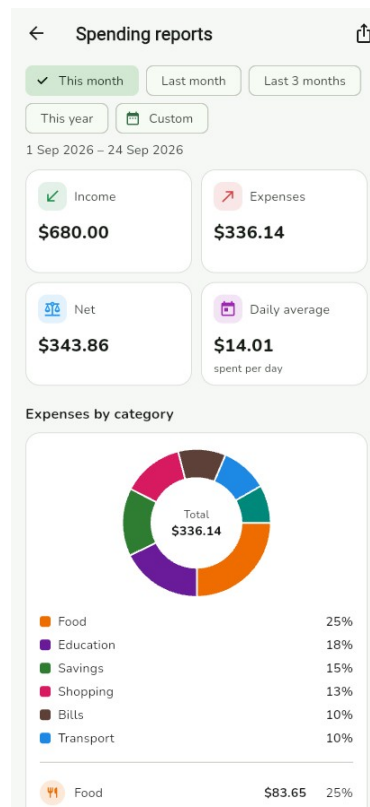


Figure 22: Spending report

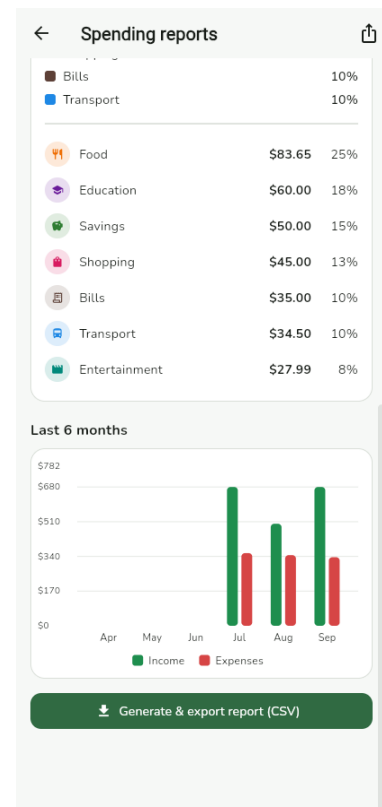


Figure 23: Category breakdown and trend

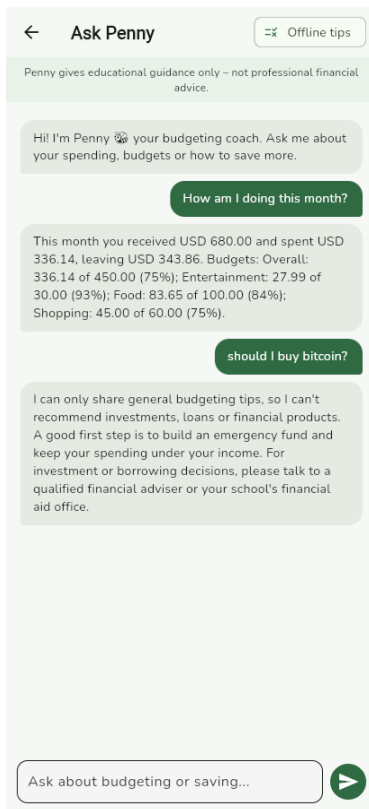


Figure 24: Ask Penny chatbot

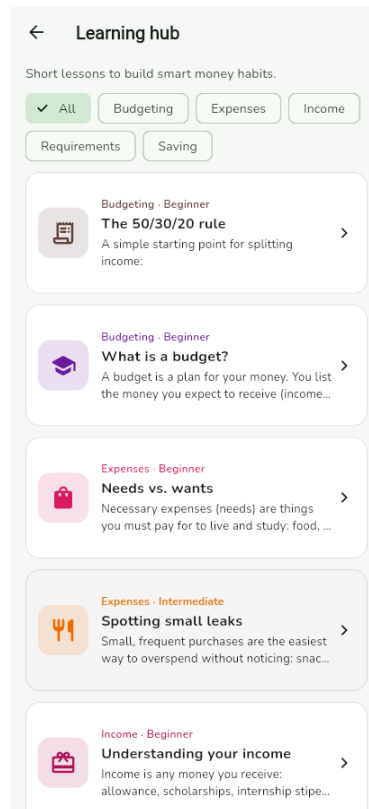


Figure 25: Learning hub

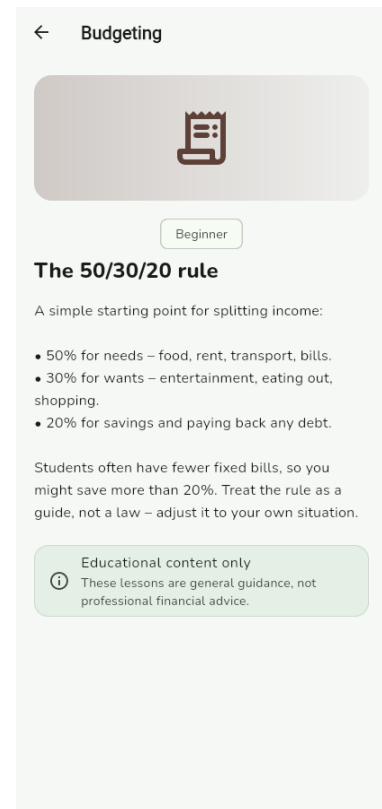


Figure 26: Lesson

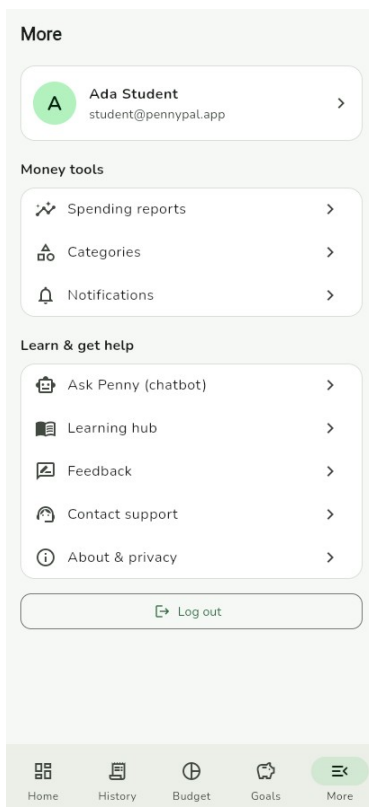


Figure 27: More menu

A.2 Admin portal

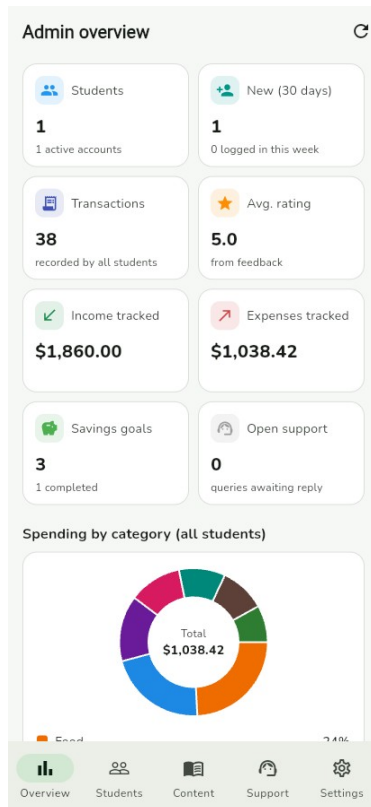


Figure 28: Admin overview

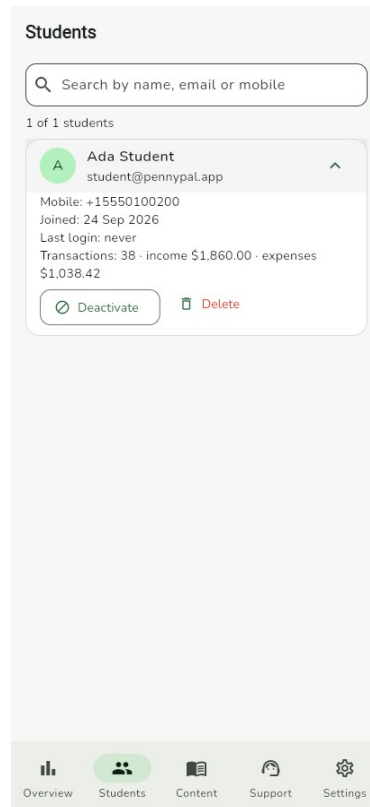


Figure 29: Student management

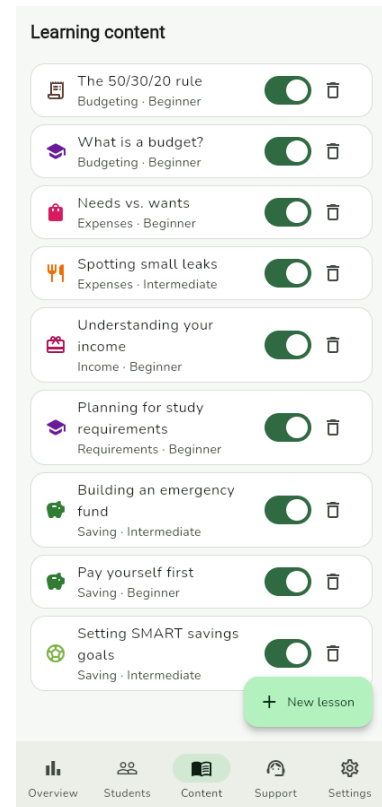


Figure 30: Learning content

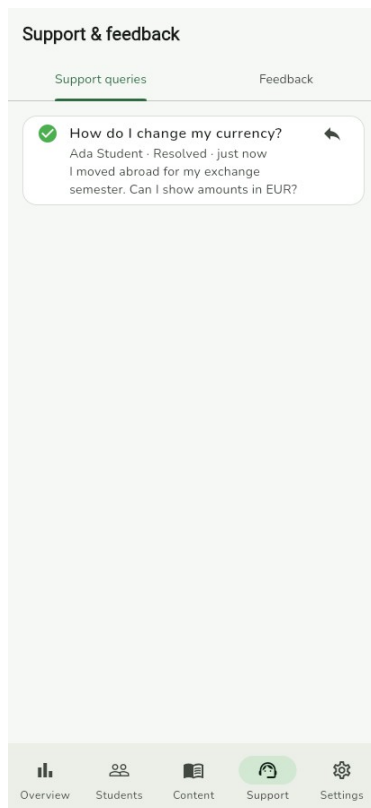


Figure 31: Support queries

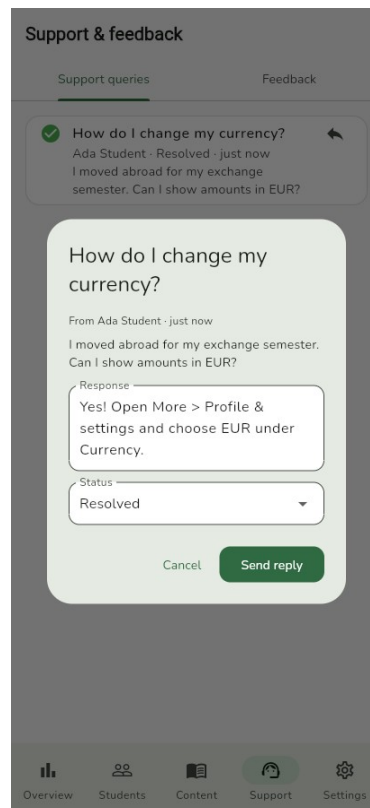


Figure 32: Replying to a query

A.3 Tablet / web layout

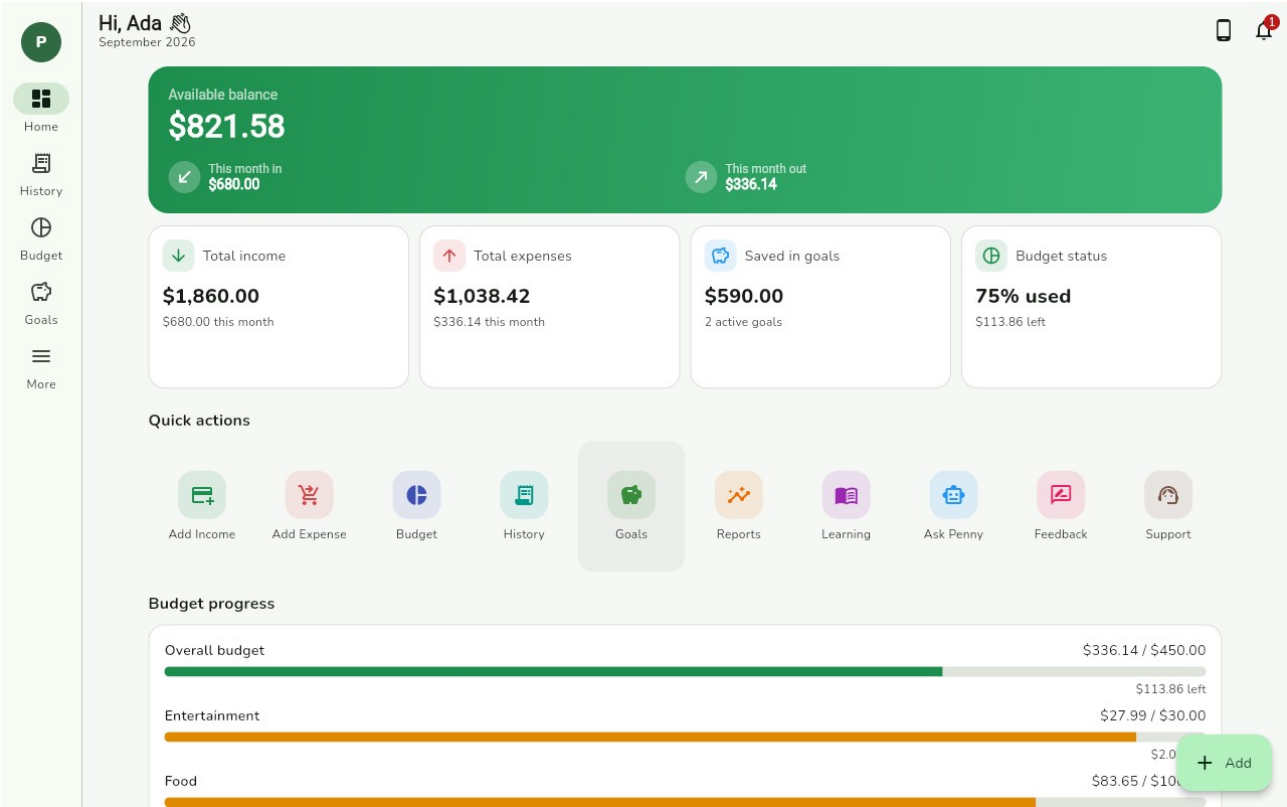


Figure 33: Dashboard on a wide screen with navigation rail